

Resolución Completa: Ejercicios de Ramificación y Poda

Algoritmia Básica

Ejercicio 1: Cámaras en el Campus

Mini Enunciado: Dado un grafo de calles y vértices, colocar el menor número de cámaras en los vértices para que todas las calles tengan al menos una cámara en uno de sus extremos.

Patrón Algorítmico: *Selección Binaria / Recubrimiento de Vértices (Vertex Cover)*. Pertenece a este patrón porque es un problema de minimización donde debemos decidir para cada elemento (vértice) si lo seleccionamos (1) o no (0) para cubrir un conjunto de restricciones (aristas).

Desarrollo Teórico:

- **a) Representación:** Una tupla $x = (x_1, \dots, x_k)$ donde $x_i \in \{0, 1\}$ indica si se coloca cámara en el vértice i . El árbol es binario, en cada nivel i se decide si $x_i = 1$ o $x_i = 0$.
- **b) Función de coste $c(x)$:** Es el número de cámaras colocadas hasta el nivel k :

$$c(x) = \sum_{i=1}^k x_i$$

- **c) Coste estimado $\hat{c}(x)$:** $\hat{c}(x) = c(x) + h(x)$. Donde $h(x)$ es el tamaño de un emparejamiento máximo en el subgrafo inducido por las aristas aún no vigiladas (es decir, el número máximo de aristas a oscuras que no comparten ningún vértice entre sí).
- **d) Función de poda $U(x)$:** Sea U el coste de la mejor solución completa encontrada. Podemos si $\hat{c}(x) \geq U$.

Código C++:

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  const int INF = 999999;
7  int N; // Num vertices
8  vector<pair<int,int>> calles;
9
10 struct Nodo {
11     int nivel, coste;
12     int cota;
13     vector<int> sol;
14     bool operator>(const Nodo& o) const { return cota > o.cota; }
15 };
16
17 int calcular_cota_camaras(const Nodo& actual) {
18     int h_x = 0;
19     vector<bool> vigilada(calles.size(), false);
20     for(size_t i=0; i<calles.size(); i++) {
21         int u = calles[i].first, v = calles[i].second;
22         if((u <= actual.nivel && actual.sol[u-1] == 1) ||
23            (v <= actual.nivel && actual.sol[v-1] == 1)) {
24             vigilada[i] = true;
25         }
26     }
27     vector<bool> vertice_usado(N + 1, false);
28     for(size_t i=0; i<calles.size(); i++) {
29         if(!vigilada[i]) {
30             int u = calles[i].first, v = calles[i].second;
31             if(!vertice_usado[u] && !vertice_usado[v]) {
32                 h_x++;
33                 vertice_usado[u] = true; vertice_usado[v] = true;
34             }
35         }
36     }
37     return h_x;
38 }
```

```

35     }
36 }
37 return actual.coste + h_x;
38 }
39
40 int ryp_camaras() {
41     priority_queue<Nodo, vector<Nodo>, greater<Nodo>> pq;
42     int mejor = INF;
43     Nodo raiz = {0, 0, 0, {}};
44     raiz.cota = calcular_cota_camaras(raiz);
45     pq.push(raiz);
46
47     while(!pq.empty()) {
48         Nodo act = pq.top(); pq.pop();
49         if(act.cota >= mejor) continue;
50
51         for(int op = 1; op >= 0; op--) { // 1 (Si), 0 (No)
52             Nodo h = act;
53             h.nivel++; h.coste += op; h.sol.push_back(op);
54             h.cota = calcular_cota_camaras(h);
55
56             if(h.nivel == N) {
57                 // Comprobar factibilidad final (todas cubiertas)
58                 bool ok = true;
59                 for(auto c : calles) {
60                     if(h.sol[c.first-1] == 0 && h.sol[c.second-1] == 0) { ok = false; break; }
61                 }
62                 if(ok && h.coste < mejor) mejor = h.coste;
63             } else if(h.cota < mejor) pq.push(h);
64         }
65     }
66     return mejor;
67 }

```

Ejercicio 2: La Pizza de Enzo

Mini Enunciado: Elegir el subconjunto mínimo de ingredientes para una pizza tal que todos los amigos tengan al menos un ingrediente de su agrado.

Patrón Algorítmico: *Selección Binaria / Recubrimiento de Conjuntos (Set Cover)*. Pertenece a este patrón porque se busca minimizar la cantidad de subconjuntos (ingredientes) elegidos de modo que su unión cubra todo el universo (amigos).

Desarrollo Teórico:

- a) **Representación:** $x = (x_1, \dots, x_k)$ donde $x_j \in \{0, 1\}$ indica si pedimos el ingrediente j .
- b) **Función de coste** $c(x)$: Ingredientes pedidos: $c(x) = \sum_{j=1}^k x_j$.
- c) **Coste estimado** $\hat{c}(x)$: $\hat{c}(x) = c(x) + \lceil P/M \rceil$, donde P es el número de amigos aún no satisfechos, y M es el número máximo de amigos insatisfechos que puede satisfacer un solo ingrediente de los que quedan por evaluar.
- d) **Función de poda** $U(x)$: Sea U el mínimo número de ingredientes de una solución completa. Podemos si $\hat{c}(x) \geq U$.

Código C++:

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <cmath>
5  using namespace std;
6
7  const int INF = 999999;
8  int N_amigos, M_ingredientes;
9  vector<vector<int>> pref; // pref[amigo][ingrediente]
10
11 struct Nodo {
12     int nivel, coste;
13     int cota, num_cubiertos;

```

```

14     vector<int> sol;
15     vector<bool> cubiertos;
16     bool operator>(const Nodo& o) const { return cota > o.cota; }
17 };
18
19 int calcular_cota_pizza(const Nodo& act) {
20     int faltan = N_amigos - act.num_cubiertos;
21     if(faltan == 0) return act.coste;
22
23     int max_cobertura = 0;
24     for(int j = act.nivel; j < M_ingredientes; j++) {
25         int cubre_este = 0;
26         for(int i = 0; i < N_amigos; i++) {
27             if(!act.cubiertos[i] && pref[i][j] == 1) cubre_este++;
28         }
29         max_cobertura = max(max_cobertura, cubre_este);
30     }
31     if(max_cobertura == 0) return INF; // Inviabile
32     return act.coste + (faltan + max_cobertura - 1) / max_cobertura;
33 }
34
35 int ryp_pizza() {
36     priority_queue<Nodo, vector<Nodo>, greater<Nodo>> pq;
37     int mejor = INF;
38     Nodo raiz = {0, 0, 0, 0, {}, vector<bool>(N_amigos, false)};
39     raiz.cota = calcular_cota_pizza(raiz);
40     pq.push(raiz);
41
42     while(!pq.empty()) {
43         Nodo act = pq.top(); pq.pop();
44         if(act.cota >= mejor) continue;
45
46         int j = act.nivel;
47         // Hijo 1: Con ingrediente
48         Nodo h1 = act; h1.nivel++; h1.coste++; h1.sol.push_back(1);
49         for(int i = 0; i < N_amigos; i++) {
50             if(pref[i][j] == 1 && !h1.cubiertos[i]) {
51                 h1.cubiertos[i] = true; h1.num_cubiertos++;
52             }
53         }
54         h1.cota = calcular_cota_pizza(h1);
55         if(h1.num_cubiertos == N_amigos && h1.coste < mejor) mejor = h1.coste;
56         else if(h1.cota < mejor && h1.nivel < M_ingredientes) pq.push(h1);
57
58         // Hijo 2: Sin ingrediente
59         Nodo h2 = act; h2.nivel++; h2.sol.push_back(0);
60         h2.cota = calcular_cota_pizza(h2);
61         if(h2.cota < mejor && h2.nivel < M_ingredientes) pq.push(h2);
62     }
63     return mejor;
64 }

```

Ejercicio 3: Aeropuertos

Mini Enunciado: Construir el mínimo número de aeropuertos para que todas las ciudades tengan uno a menos de 100 km.

Patrón Algorítmico: *Selección Binaria / Recubrimiento de Conjuntos (Set Cover)*. Es matemáticamente idéntico al problema de la pizza. Sustituimos ingredientes por aeropuertos y amigos por ciudades.

Desarrollo Teórico:

- a) **Representación:** $x = (x_1, \dots, x_k)$ donde $x_i \in \{0, 1\}$ indica si se construye aeropuerto en la ciudad i .
- b) **Función de coste** $c(x)$: $c(x) = \sum_{i=1}^k x_i$.
- c) **Coste estimado** $\hat{c}(x)$: $\hat{c}(x) = c(x) + \lceil (\text{ciudades sin cubrir}) / (\text{max cobertura de un aeropuerto restante}) \rceil$.
- d) **Función de poda** $U(x)$: Poda si $\hat{c}(x) \geq U$.

Código C++: (La estructura es idéntica al Ejercicio 2, omitida para evitar redundancia extrema, basta con usar la matriz de adyacencia de distancias < 100 km en lugar de la matriz de preferencias).

Ejercicio 4: Asignación de Personas a Tareas

Mini Enunciado: Asignar n personas a n tareas maximizando el beneficio, pero resolviéndolo matemáticamente como un problema de minimización.

Patrón Algorítmico: *Emparejamiento / Asignación 1 a 1 (Transformación Max a Min)*. Pertenece al patrón de matrices donde se asignan elementos de forma biunívoca. Transformamos la matriz restando el máximo global a cada elemento para usar la heurística de mínimo coste.

Desarrollo Teórico:

- **a) Representación:** $x = (x_1, \dots, x_k)$ donde x_i es el índice de la tarea asignada a la persona i .
- **b) Función de coste $c(x)$:** Suma de los costes transformados: $c(x) = \sum_{i=1}^k C[i][x_i]$ (donde $C[i][j] = \text{Max} - B[i][j]$).
- **c) Coste estimado $\hat{c}(x)$:** $\hat{c}(x) = c(x) + \sum_{j=k+1}^N \text{mín}(\text{valores libres fila } j)$.
- **d) Función de poda $U(x)$:** Sea U el coste mínimo encontrado. Poda si $\hat{c}(x) \geq U$. Al final, Beneficio $= (N \times \text{Max}) - U$.

Código C++:

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  const int INF = 999999;
7  int N;
8  vector<vector<int>> B; // Beneficios
9  vector<vector<int>> C; // Costes transformados
10
11 struct Nodo {
12     int nivel, coste, cota;
13     vector<int> sol;
14     vector<bool> usada;
15     bool operator>(const Nodo& o) const { return cota > o.cota; }
16 };
17
18 int calcular_cota_asig(const Nodo& act) {
19     int h_x = 0;
20     for(int i = act.nivel; i < N; i++) {
21         int min_val = INF;
22         for(int j = 0; j < N; j++) {
23             if(!act.usada[j] && C[i][j] < min_val) min_val = C[i][j];
24         }
25         h_x += min_val;
26     }
27     return act.coste + h_x;
28 }
29
30 int ryp_asignacion() {
31     int max_ben = 0;
32     for(int i=0; i<N; i++)
33         for(int j=0; j<N; j++)
34             max_ben = max(max_ben, B[i][j]);
35
36     C.assign(N, vector<int>(N, 0));
37     for(int i=0; i<N; i++)
38         for(int j=0; j<N; j++)
39             C[i][j] = max_ben - B[i][j];
40
41     priority_queue<Nodo, vector<Nodo>, greater<Nodo>> pq;
42     int mejor = INF;
43     Nodo raiz = {0, 0, 0, vector<int>(N, -1), vector<bool>(N, false)};
44     raiz.cota = calcular_cota_asig(raiz);
45     pq.push(raiz);
46
47     while(!pq.empty()) {
48         Nodo act = pq.top(); pq.pop();
49         if(act.cota >= mejor) continue;
50
51         int p = act.nivel;
52         for(int t = 0; t < N; t++) {
53             if(!act.usada[t]) {
```

```

54         Nodo h = act;
55         h.nivel++; h.coste += C[p][t]; h.sol[p] = t; h.usada[t] = true;
56         h.cota = calcular_cota_asig(h);
57
58         if(h.nivel == N && h.coste < mejor) mejor = h.coste;
59         else if(h.cota < mejor) pq.push(h);
60     }
61 }
62 }
63 return (N * max_ben) - mejor;
64 }

```

Ejercicio 5: El Tira y Afloja

Mini Enunciado: Dividir a n personas en dos equipos de modo que la diferencia de integrantes no sea mayor a 1, y la diferencia de peso total sea mínima.

Patrón Algorítmico: *Partición de Subconjuntos / Mochila Binaria*. Pertenece a este patrón porque es una decisión binaria (Equipo 0 o Equipo 1) buscando minimizar la diferencia entre dos acumuladores bajo restricciones de capacidad.

Desarrollo Teórico:

- **a) Representación:** $x = (x_1, \dots, x_k) \in \{0, 1\}^k$. 0=Equipo A, 1=Equipo B.
- **b) Función de coste $c(x)$:** Solo evaluable en las hojas (nivel n): $c(x) = |W_A - W_B|$. En nodos internos no hay coste acumulado per se.
- **c) Coste estimado $\hat{c}(x)$:** $\hat{c}(x) = 0$ si las sumas actuales permiten igualarse perfectamente con los pesos restantes. Si un equipo ya supera la mitad del peso total, $\hat{c}(x) = W_{mayor} - (W_{menor} + \text{Restantes})$.
- **d) Función de poda $U(x)$:** Sea U la mínima diferencia encontrada. 1) Poda de viabilidad: Cortar si $N_A > \lceil n/2 \rceil$ o $N_B > \lceil n/2 \rceil$. 2) Poda por cota: Cortar si $\hat{c}(x) \geq U$.

Código C++:

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <cmath>
5  #include <numeric>
6  using namespace std;
7
8  const int INF = 9999999;
9  int N;
10 vector<int> pesos;
11
12 struct Nodo {
13     int nivel, w0, w1, n0, n1, cota;
14     bool operator>(const Nodo& o) const { return cota > o.cota; }
15 };
16
17 int calcular_cota_tira(const Nodo& act, int w_total) {
18     int w_restante = w_total - (act.w0 + act.w1);
19     int mayor = max(act.w0, act.w1);
20     int menor = min(act.w0, act.w1);
21     if (mayor > menor + w_restante) return mayor - (menor + w_restante);
22     return 0; // Podrian llegar a igualarse
23 }
24
25 int ryp_tira_afloja() {
26     int w_total = accumulate(pesos.begin(), pesos.end(), 0);
27     int limite_p = (N + 1) / 2;
28
29     priority_queue<Nodo, vector<Nodo>, greater<Nodo>> pq;
30     int mejor = INF;
31     Nodo raiz = {0, 0, 0, 0, 0, 0};
32     raiz.cota = calcular_cota_tira(raiz, w_total);
33     pq.push(raiz);
34
35     while(!pq.empty()) {

```

```

36     Nodo act = pq.top(); pq.pop();
37     if(act.cota >= mejor) continue;
38
39     int p_peso = pesos[act.nivel];
40
41     // Opcion 0
42     if(act.n0 + 1 <= limite_p) {
43         Nodo h0 = act; h0.nivel++; h0.w0 += p_peso; h0.n0++;
44         h0.cota = calcular_cota_tira(h0, w_total);
45         if(h0.nivel == N && abs(h0.w0 - h0.w1) < mejor) mejor = abs(h0.w0 - h0.w1);
46         else if(h0.cota < mejor) pq.push(h0);
47     }
48
49     // Opcion 1
50     if(act.n1 + 1 <= limite_p) {
51         Nodo h1 = act; h1.nivel++; h1.w1 += p_peso; h1.n1++;
52         h1.cota = calcular_cota_tira(h1, w_total);
53         if(h1.nivel == N && abs(h1.w0 - h1.w1) < mejor) mejor = abs(h1.w0 - h1.w1);
54         else if(h1.cota < mejor) pq.push(h1);
55     }
56 }
57 return mejor;
58 }

```

Ejercicio 6: Menús del Banquete y Ejercicio 7: Animales

Nota sobre Patrones: El **Ejercicio 6** es matemáticamente idéntico al Ejercicio 2 y 3 (*Set Cover*). Se pide minimizar un subconjunto de vectores binarios que cubran a todos los individuos. La formulación de las cotas y el árbol es la misma.

El **Ejercicio 7** es matemáticamente idéntico al Ejercicio 4 (*Asignación 1 a 1*). Se trata de asignar niños a animales buscando la satisfacción máxima, aplicando el mismo truco de restar al valor máximo para transformarlo en un problema de minimización.

Ejercicio 8: Diseño del Periódico

Mini Enunciado: Elegir qué artículos colocar en una página de periódico dadas sus dimensiones y coordenadas fijas, minimizando el espacio libre (sin que se solapen ni se salgan del papel).

Patrón Algorítmico: *Empaquetamiento Espacial / Selección 2D*. Sigue el patrón binario de la mochila (poner artículo o no), pero la viabilidad física la dictan restricciones geométricas 2D en lugar de una suma simple.

Desarrollo Teórico:

- **a) Representación:** $x = (x_1, \dots, x_k) \in \{0, 1\}^k$. Indica si se imprime el artículo i .
- **b) Función de coste $c(x)$:** Espacio libre sin poner nada más: $c(x) = (W \times L) - \text{Área_Ocupada}$.
- **c) Coste estimado $\hat{c}(x)$:** Se asume el relleno perfecto utópico: $\hat{c}(x) = \max(0, (W \times L) - (\text{Área_Ocupada} + \text{Suma_Áreas_Restantes}))$.
- **d) Función de poda $U(x)$:** 1) Poda física: Si el artículo choca con otro existente o se sale del papel. 2) Poda por cota: Sea U el récord de mínimo espacio libre, cortar si $\hat{c}(x) \geq U$.

Código C++:

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  const int INF = 999999999;
7  struct Art { int x, y, w, h; int area() const { return w*h; } };
8  int W, L;
9  vector<Art> arts;

```

```

10
11 struct Nodo {
12     int nivel, area_oc, cota;
13     vector<int> sol;
14     bool operator>(const Nodo& o) const { return cota > o.cota; }
15 };
16
17 bool chocan(const Art& A, const Art& B) {
18     if (B.x >= A.x + A.w) return false;
19     if (B.x + B.w <= A.x) return false;
20     if (B.y >= A.y + A.h) return false;
21     if (B.y + B.h <= A.y) return false;
22     return true;
23 }
24
25 int calc_cota_per(const Nodo& act) {
26     int ideal = 0;
27     for(size_t i = act.nivel; i < arts.size(); i++) ideal += arts[i].area();
28     return max(0, (W * L) - (act.area_oc + ideal));
29 }
30
31 int ryp_periodico() {
32     priority_queue<Nodo, vector<Nodo>, greater<Nodo>> pq;
33     int mejor = INF;
34     Nodo raiz = {0, 0, 0, {}};
35     raiz.cota = calc_cota_per(raiz);
36     pq.push(raiz);
37
38     while(!pq.empty()) {
39         Nodo act = pq.top(); pq.pop();
40         if(act.cota >= mejor) continue;
41
42         int i = act.nivel;
43         Art art_i = arts[i];
44
45         // Hijo 1: Poner
46         bool cabe = (art_i.x + art_i.w <= W) && (art_i.y + art_i.h <= L);
47         if(cabe) {
48             bool colision = false;
49             for(int j=0; j<i; j++) {
50                 if(act.sol[j] == 1 && chocan(art_i, arts[j])) { colision = true; break; }
51             }
52             if(!colision) {
53                 Nodo h1 = act; h1.nivel++; h1.area_oc += art_i.area(); h1.sol.push_back(1);
54                 h1.cota = calc_cota_per(h1);
55                 if(h1.nivel == arts.size() && (W*L - h1.area_oc) < mejor) mejor = (W*L - h1.
                    area_oc);
56                 else if(h1.cota < mejor) pq.push(h1);
57             }
58         }
59
60         // Hijo 2: No poner
61         Nodo h2 = act; h2.nivel++; h2.sol.push_back(0);
62         h2.cota = calc_cota_per(h2);
63         if(h2.nivel == arts.size() && (W*L - h2.area_oc) < mejor) mejor = (W*L - h2.area_oc);
64         else if(h2.cota < mejor) pq.push(h2);
65     }
66     return mejor;
67 }

```